



VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

**School of Computer Science Engineering and Information Systems
(SCORE)**

Winter Semester 2025-2026

**ISWE406P – AGILE DEVELOPMENT PROCESS AND
DEVOPS LAB**

JENKINS TASK SHEET

CI TOOL: WOODPEKER CI

NAME OF THE STUDENT: VISHAL R

REG NO OF THE STUDENT: 23MIS0133

PROGRAMME: M.Tech-SOFTWARE ENGINEERING

SLOT: L51+L52

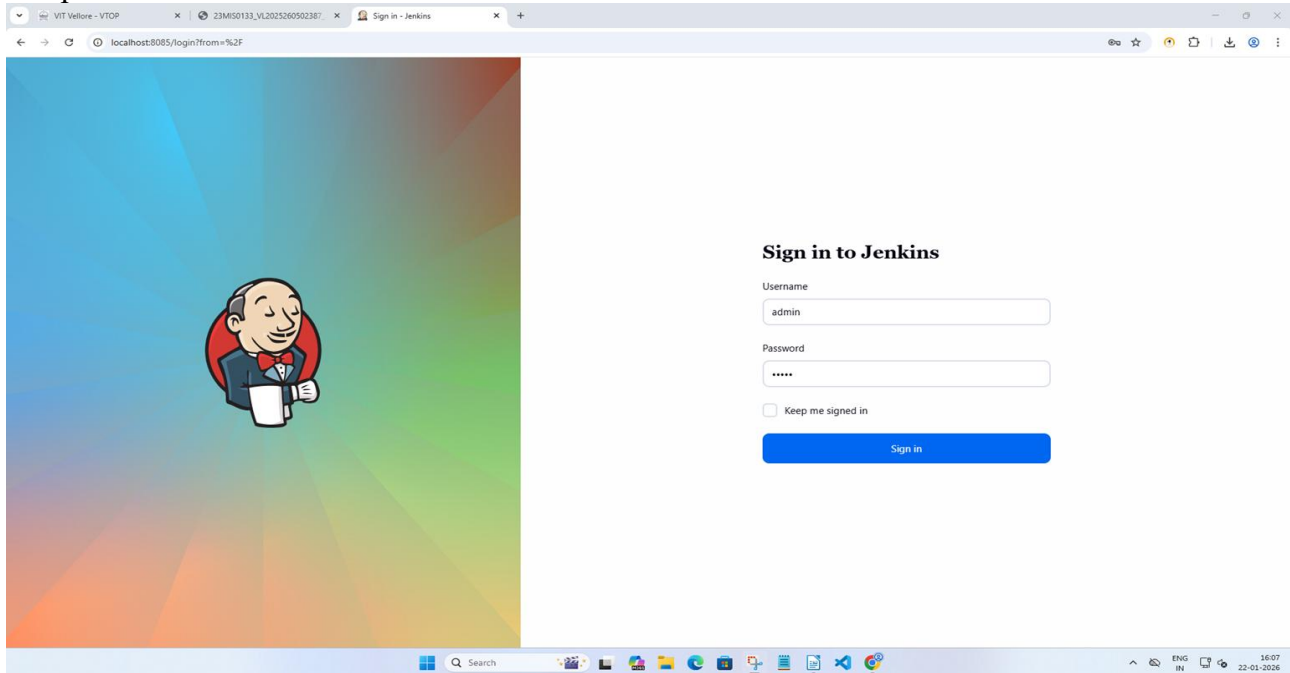
FACULTY NAME: Prof. KALAIVANI S

TASK 1:

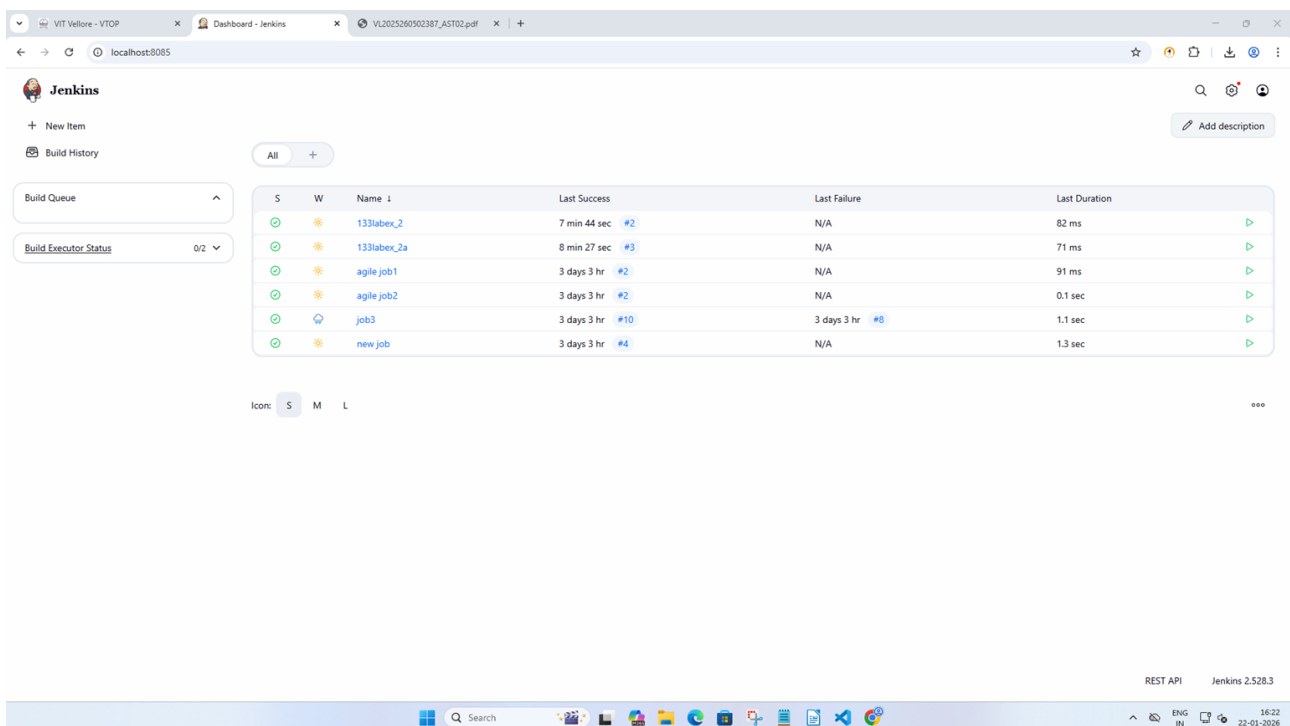
Jenkins Familiarization Objective: Understand Jenkins UI and basic navigation

Tasks:

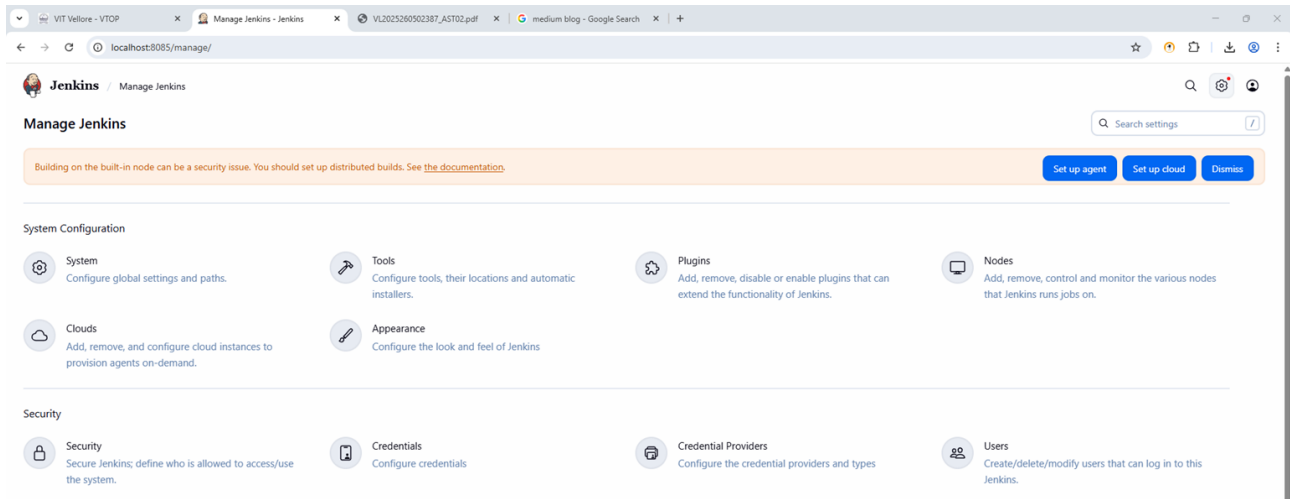
1. Open Jenkins Dashboard in browser



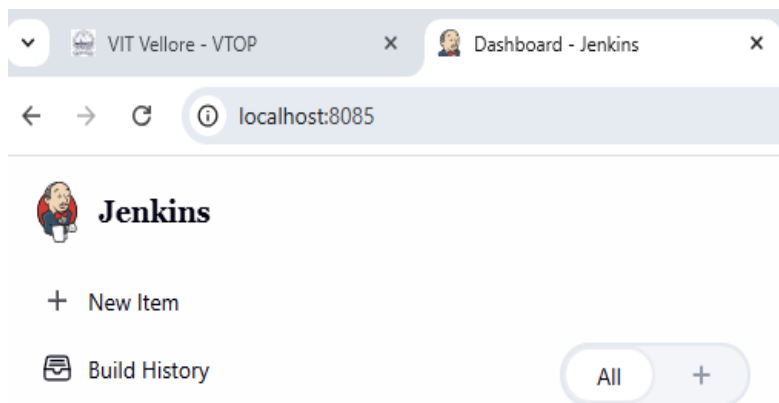
2. Identify: o Dashboard



o Manage Jenkins



o New Item o Build History

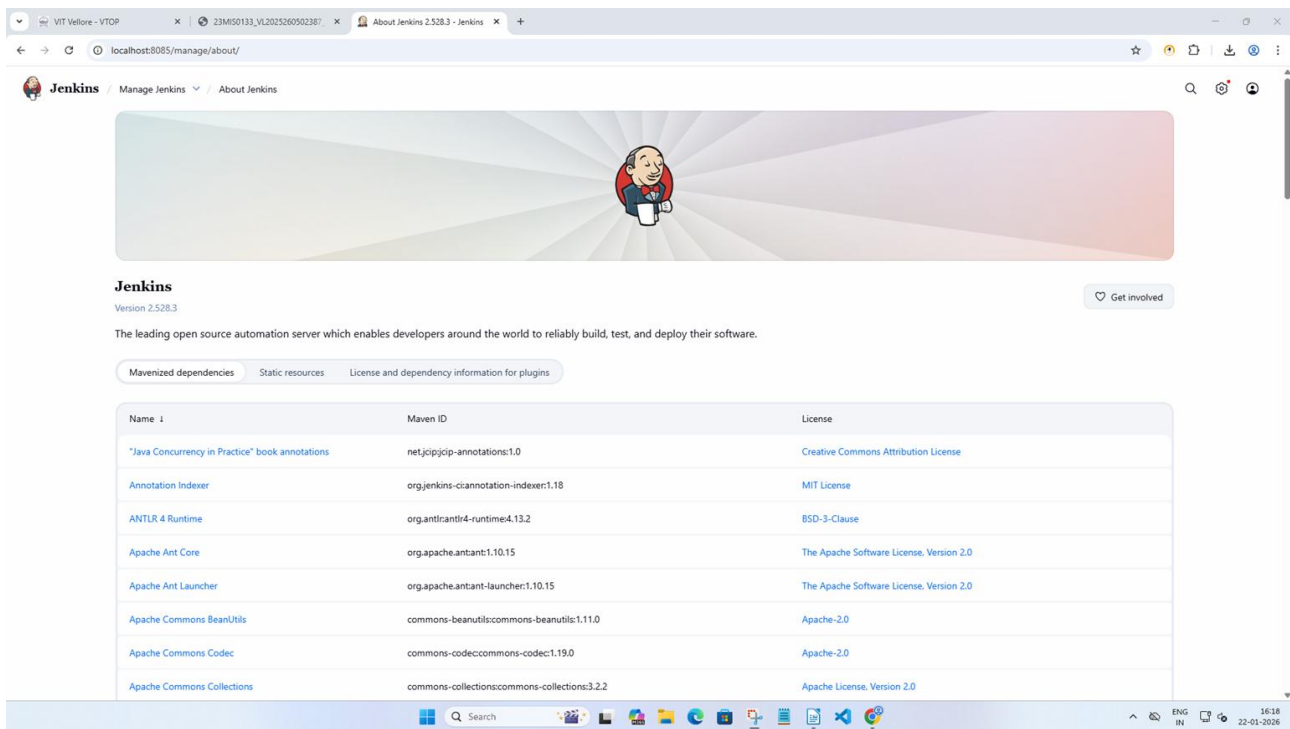


3. Check Jenkins version

Jenkins

Version 2.528.3

The leading open source automation server which enables developers around the world to reliably build, test, and deploy their software.



TASK 2:

Create First Freestyle Job Objective: Create and run a Jenkins job

Tasks:

1. Create a Freestyle project named Hello-Jenkins

New Item

Enter an item name

Hello-Jenkins

Select an item type



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.



Multibranch Pipeline

Creates a set of Pipeline projects according to detected branches in one SCM repository.



Organization Folder

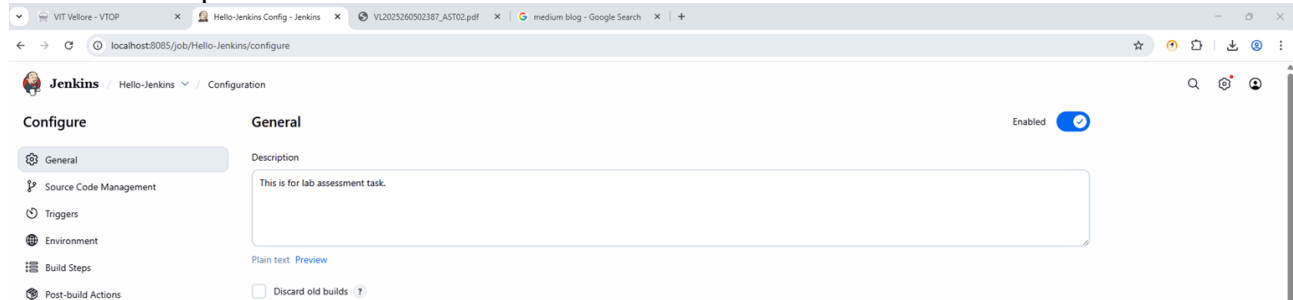
Creates a set of multibranch project subfolders by scanning for repositories.

If you want to create a new item from other existing, you can use this option:

Copy from

OK

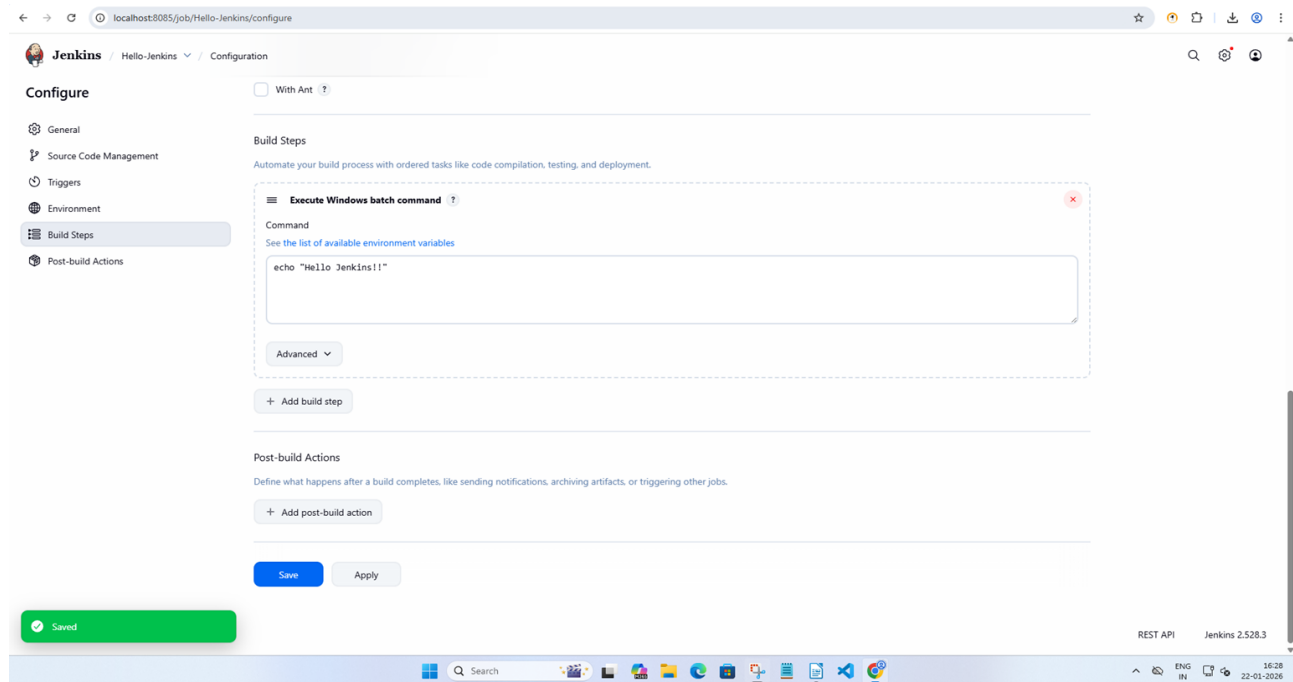
2. Add a description



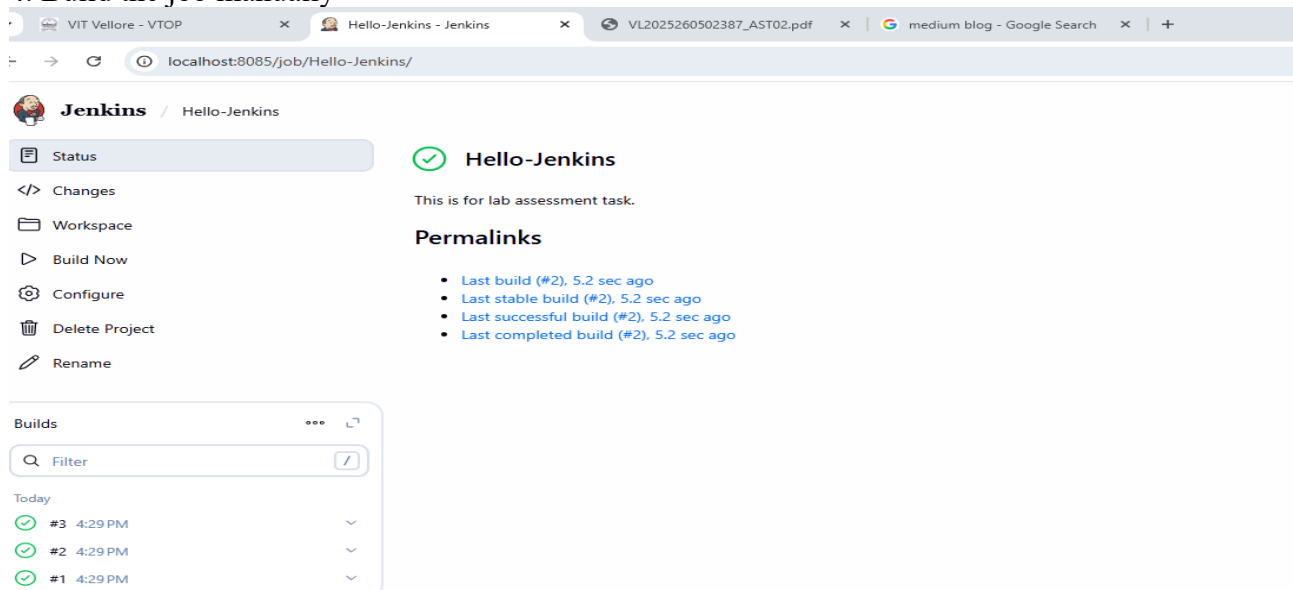
3. Add build step:

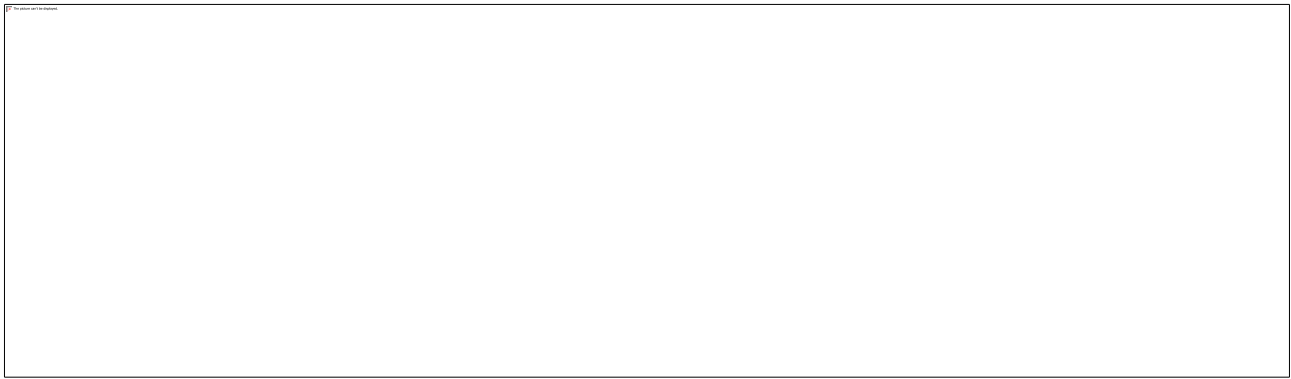
o Execute shell / Windows batch command

o Print "Hello Jenkins"



4. Build the job manually





TASK 3:

Jenkins Workspace & Commands Objective: Understand workspace usage

Tasks:

1. Navigate to job workspace

The screenshot shows the Jenkins web interface in a browser. The address bar indicates the URL is `localhost:8085/job/Hello-Jenkins/ws/`. The page title is "Workspace of Hello-Jenkins on Built-In Node". On the left sidebar, the "Workspace" tab is selected. The main content area shows "No files in directory". Below the sidebar, there is a "Builds" section with a filter input and a list of three builds, all marked as successful with green checkmarks and timestamped "4:29 PM".

Jenkins / Hello-Jenkins / Workspace of Hello-Jenkins...

Workspace of Hello-Jenkins on Built-In Node

Hello-Jenkins / →

No files in directory

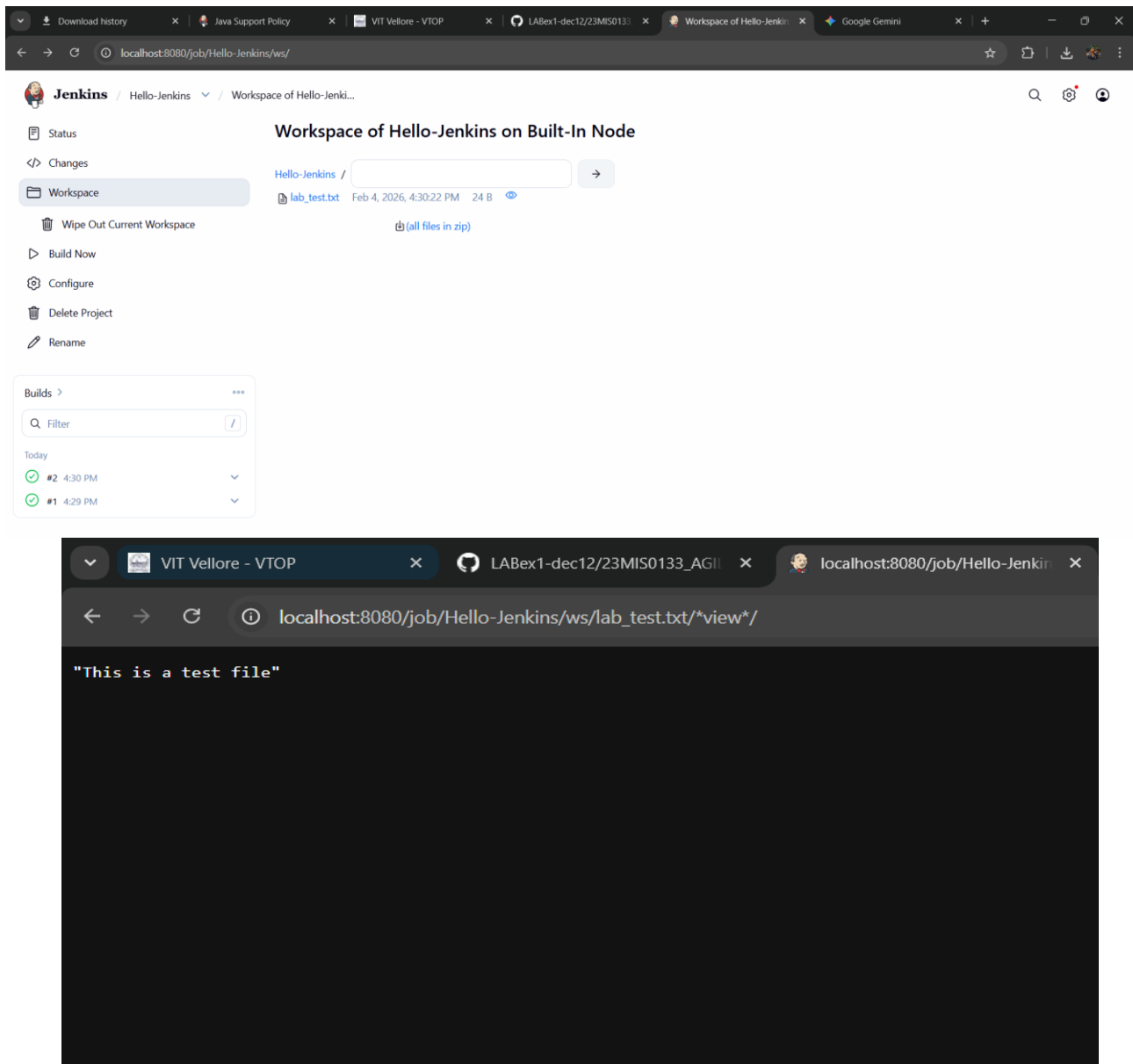
Builds

Filter

Today

- ✓ #3 4:29 PM
- ✓ #2 4:29 PM
- ✓ #1 4:29 PM

2. Create a text file using build step



3. Display file contents in console

```
Started by user VISHAL
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\.jenkins\workspace\Hello-Jenkins
[Hello-Jenkins] $ cmd /c call C:\windows\TEMP\jenkins10510333439432831146.bat

C:\ProgramData\Jenkins\.jenkins\workspace\Hello-Jenkins>echo "This is a test file" 1>lab_test.txt

C:\ProgramData\Jenkins\.jenkins\workspace\Hello-Jenkins>type lab_test.txt
"This is a test file"

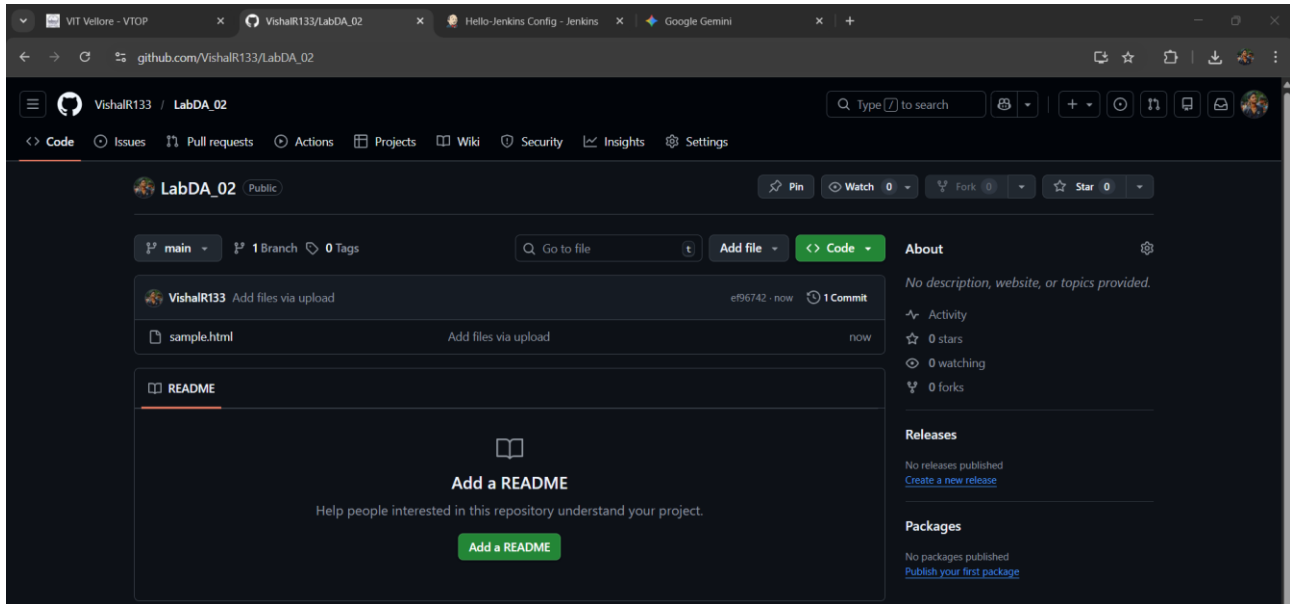
C:\ProgramData\Jenkins\.jenkins\workspace\Hello-Jenkins>exit 0
Finished: SUCCESS
```

TASK 4:

Git Integration Objective: Integrate Jenkins with GitHub

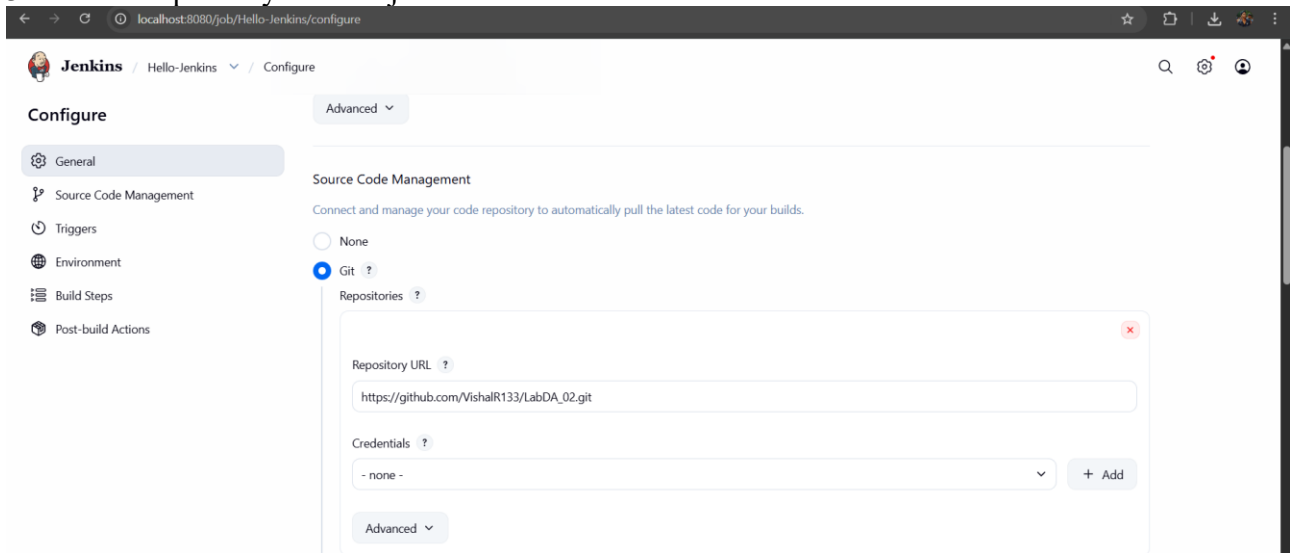
Tasks:

1. Create a GitHub repository with sample code



2. Configure Git in Jenkins

3. Add Git repository URL in job



4. Build and verify code checkout

The screenshot shows the Jenkins web interface. The top part displays the console output for a build named 'Hello-Jenkins'.

```
Started by user VISHAL
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins
The recommended git tool is: NONE
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/VishalR133/LabDA_02.git # timeout=10
Fetching upstream changes from https://github.com/VishalR133/LabDA_02.git
> git.exe --version # timeout=10
> git --version # 'git version 2.52.0.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/VishalR133/LabDA_02.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision ef967420c79155c070e307fec59ef258cdc51a18 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f ef967420c79155c070e307fec59ef258cdc51a18 # timeout=10
Commit message: "Add files via upload"
First time build. Skipping changelog.
[Hello-Jenkins] $ cmd /c call C:\windows\TEMP\jenkins6294571596054960994.bat

C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins>echo "This is a test file" >lab_test.txt

C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins>type lab_test.txt
"This is a test file"

C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins>exit 0
Finished: SUCCESS
```

The bottom part shows the 'Workspace of Hello-Jenkins on Built-In Node' view. It lists files in the workspace:

File	Size	Modified
lab_test.txt	24 B	Feb 4, 2026, 5:05:44 PM
sample.html	246 B	Feb 4, 2026, 5:05:40 PM

The browser address bar shows the URL: `localhost:8080/job/Hello-Jenkins/ws/sample.html`.

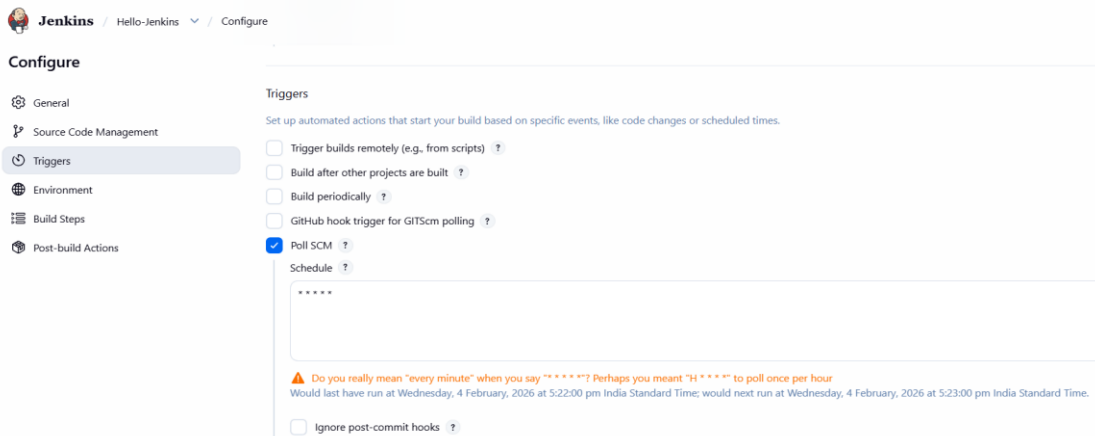
This is for lab DA 02!!

TASK 5:

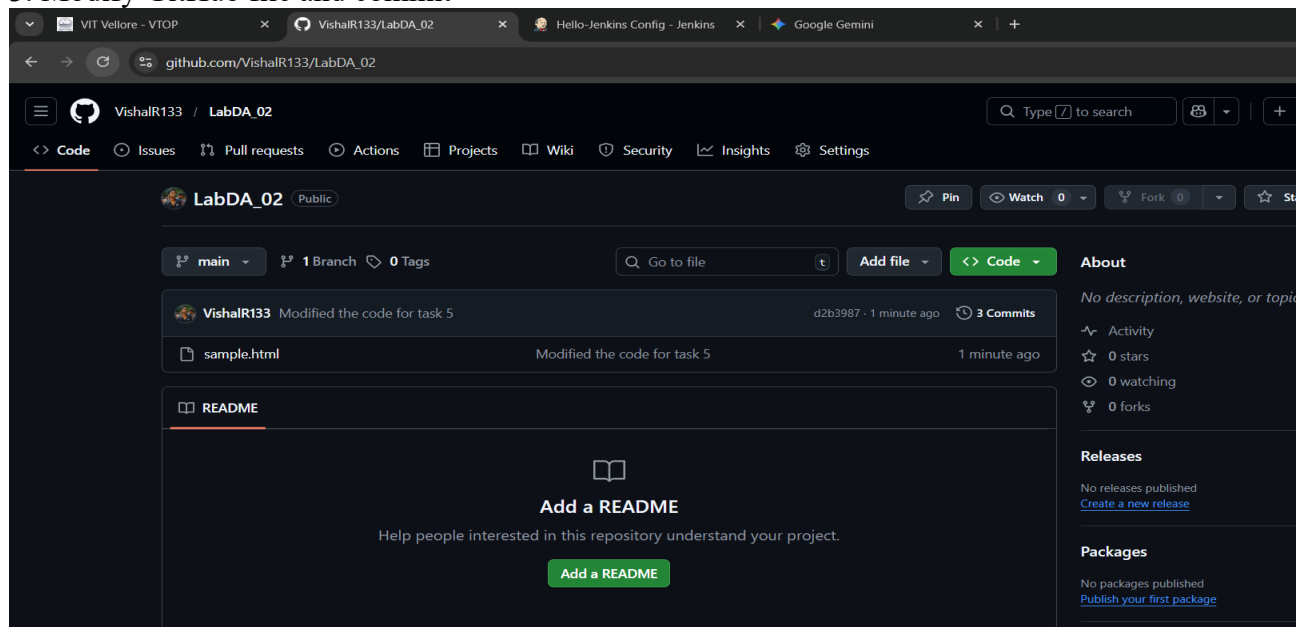
Poll SCM Trigger Objective: Automatically trigger builds on code change

Tasks:

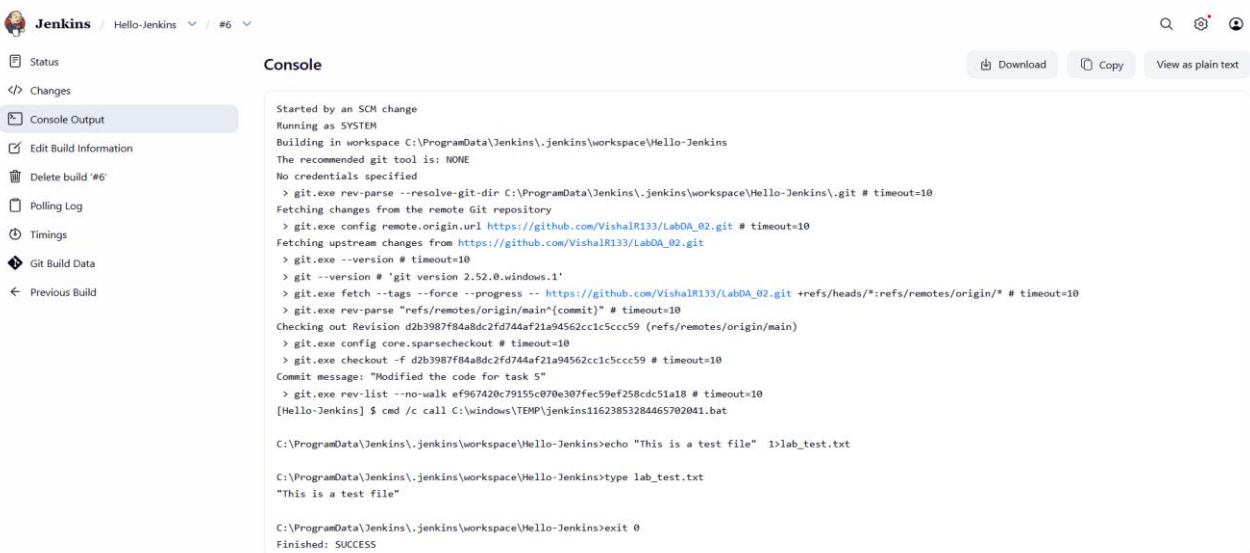
1. Enable Poll SCM
2. Set schedule: * * * * *

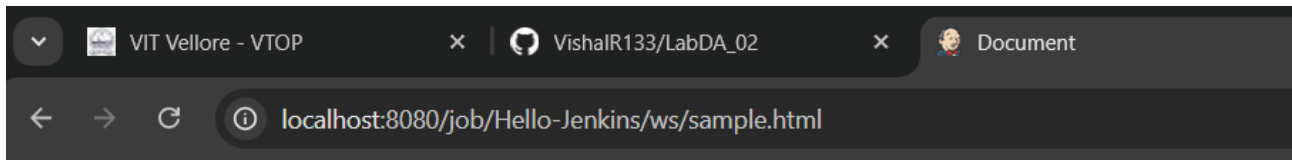


3. Modify GitHub file and commit



4. Observe automatic build





This is for lab DA 02!!

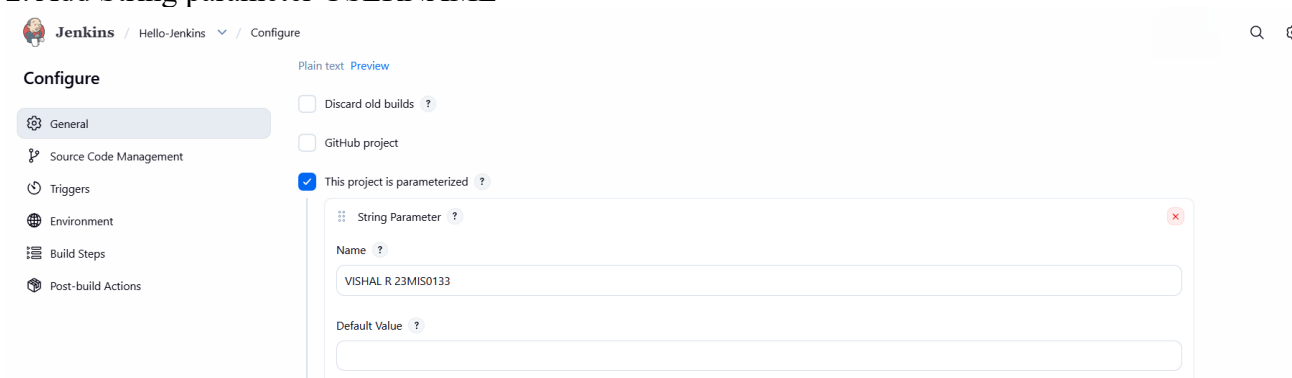
Task 5: Automatic trigger!!

TASK 6:

Parameterized Build Objective: Use parameters in Jenkins job

Tasks:

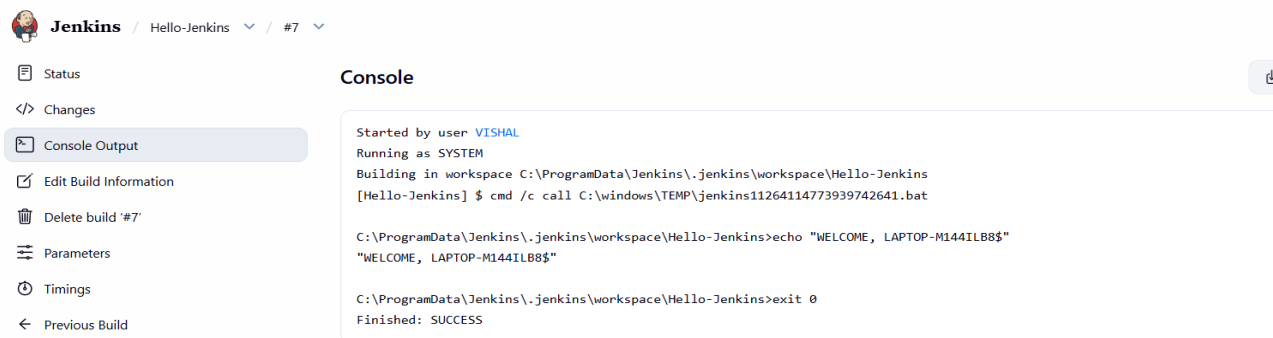
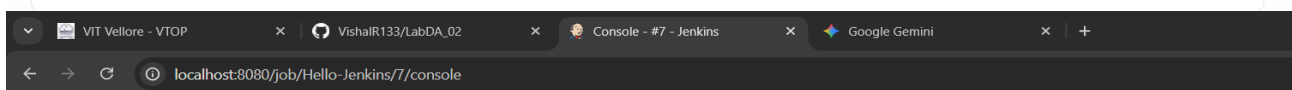
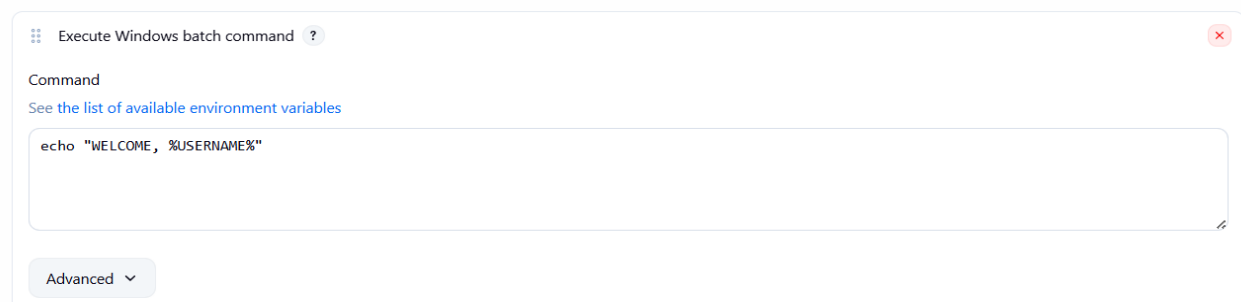
1. Enable parameterized build
2. Add String parameter USERNAME



3. Print parameter value in build step

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.



3. Run Java program

The screenshot shows the Jenkins console output for build #9. The left sidebar contains links for Status, Changes, Console Output (selected), Edit Build Information, Delete build '#9', Timings, Git Build Data, and Previous Build. The console output shows the following steps:

```
Started by user VISHAL
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins
The recommended git tool is: NONE
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/VishalR133/LabDA_02.git # timeout=10
Fetching upstream changes from https://github.com/VishalR133/LabDA_02.git
> git.exe --version # timeout=10
> git --version # 'git version 2.52.0.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/VishalR133/LabDA_02.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse "refs/remotes/origin/main^(commit)" # timeout=10
Checking out Revision 2a3f0d5592621477eeb5b34c1563da49232a4879 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f 2a3f0d5592621477eeb5b34c1563da49232a4879 # timeout=10
Commit message: "Added new file for Task 7"
> git.exe rev-list --no-walk d2b3987f84a8dc2fd744af21a94562cc1c5ccc59 # timeout=10
[Hello-Jenkins] $ cmd /c call C:\windows\TEMP\Jenkins16181681614752123481.bat

C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins>javac Hello.java

C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins>java Hello
Hello Jenkins! Task 7 is successful.

C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins>exit 0
Finished: SUCCESS
```

TASK 8: Archive Artifacts Objective: Store build outputs

Tasks:

1. Generate .class or .jar file

The screenshot shows the Jenkins configuration page for build #10. The left sidebar contains links for General, Source Code Management, Triggers, Environment, Build Steps (selected), and Post-build Actions. The main content area shows the 'Execute Windows batch command' step with the following command:

```
javac Hello.java
java Hello
```

Below the command field, there is a section for 'Post-build Actions' with the following configuration:

- Archive the artifacts
- Files to archive: `**/*.class`

The 'Save' button is highlighted in blue.

2. Archive artifacts in post-build action

The screenshot shows the Jenkins console output for build #10. The left sidebar contains links for Status, Changes, Console Output (selected), Edit Build Information, Delete build '#10', Timings, Git Build Data, and Previous Build. The console output shows the following steps:

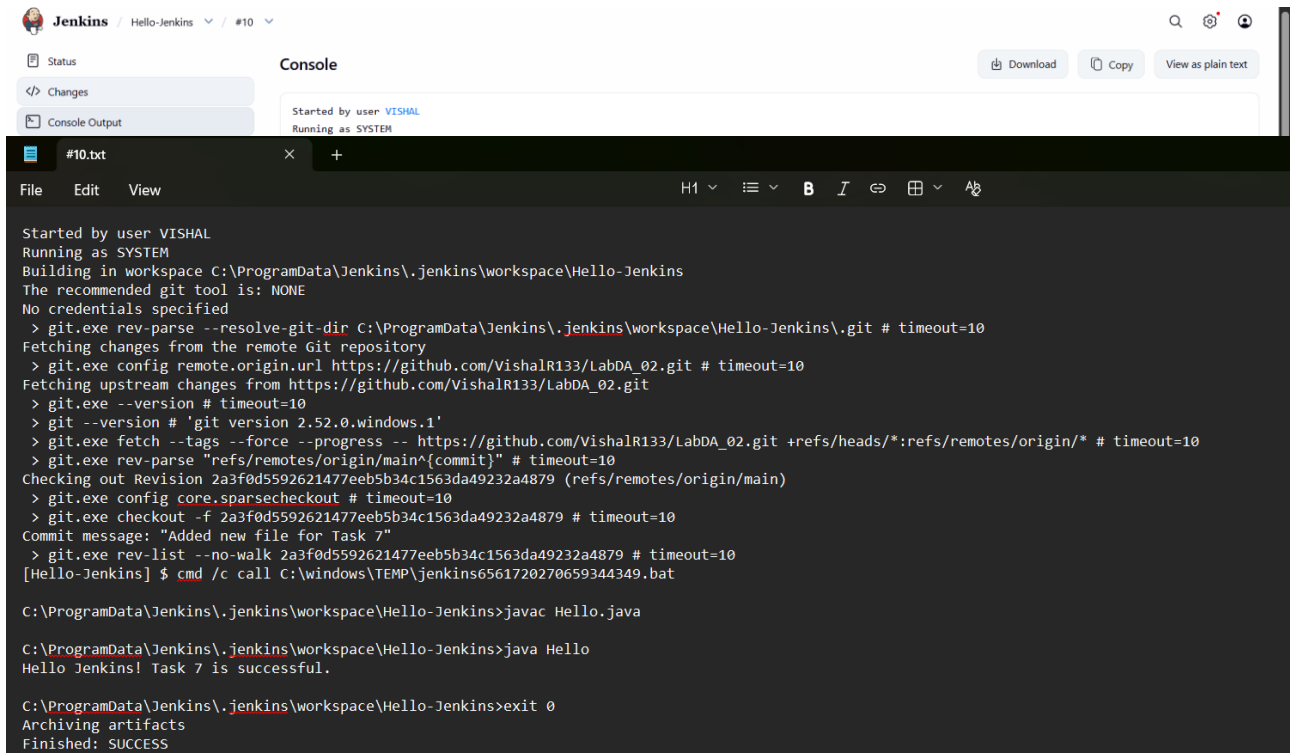
```
Started by user VISHAL
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins
The recommended git tool is: NONE
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/VishalR133/LabDA_02.git # timeout=10
Fetching upstream changes from https://github.com/VishalR133/LabDA_02.git
> git.exe --version # timeout=10
> git --version # 'git version 2.52.0.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/VishalR133/LabDA_02.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse "refs/remotes/origin/main^(commit)" # timeout=10
Checking out Revision 2a3f0d5592621477eeb5b34c1563da49232a4879 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f 2a3f0d5592621477eeb5b34c1563da49232a4879 # timeout=10
Commit message: "Added new file for Task 7"
> git.exe rev-list --no-walk 2a3f0d5592621477eeb5b34c1563da49232a4879 # timeout=10
[Hello-Jenkins] $ cmd /c call C:\windows\TEMP\Jenkins6561720270659344349.bat

C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins>javac Hello.java

C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins>java Hello
Hello Jenkins! Task 7 is successful.

C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins>exit 0
Archiving artifacts
Finished: SUCCESS
```

3. Download artifact from Jenkins UI



The screenshot shows the Jenkins console output for a build named 'Hello-Jenkins'. The build was started by user 'VISHAL' and is running as 'SYSTEM'. The console output shows the following steps:

```
Started by user VISHAL
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins
The recommended git tool is: NONE
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/VishalR133/LabDA_02.git # timeout=10
Fetching upstream changes from https://github.com/VishalR133/LabDA_02.git
> git.exe --version # timeout=10
> git --version # 'git version 2.52.0.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/VishalR133/LabDA_02.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision 2a3f0d5592621477eeb5b34c1563da49232a4879 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f 2a3f0d5592621477eeb5b34c1563da49232a4879 # timeout=10
Commit message: "Added new file for Task 7"
> git.exe rev-list --no-walk 2a3f0d5592621477eeb5b34c1563da49232a4879 # timeout=10
[Hello-Jenkins] $ cmd /c call C:\windows\TEMP\jenkins6561720270659344349.bat

C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins>javac Hello.java

C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins>java Hello
Hello Jenkins! Task 7 is successful.

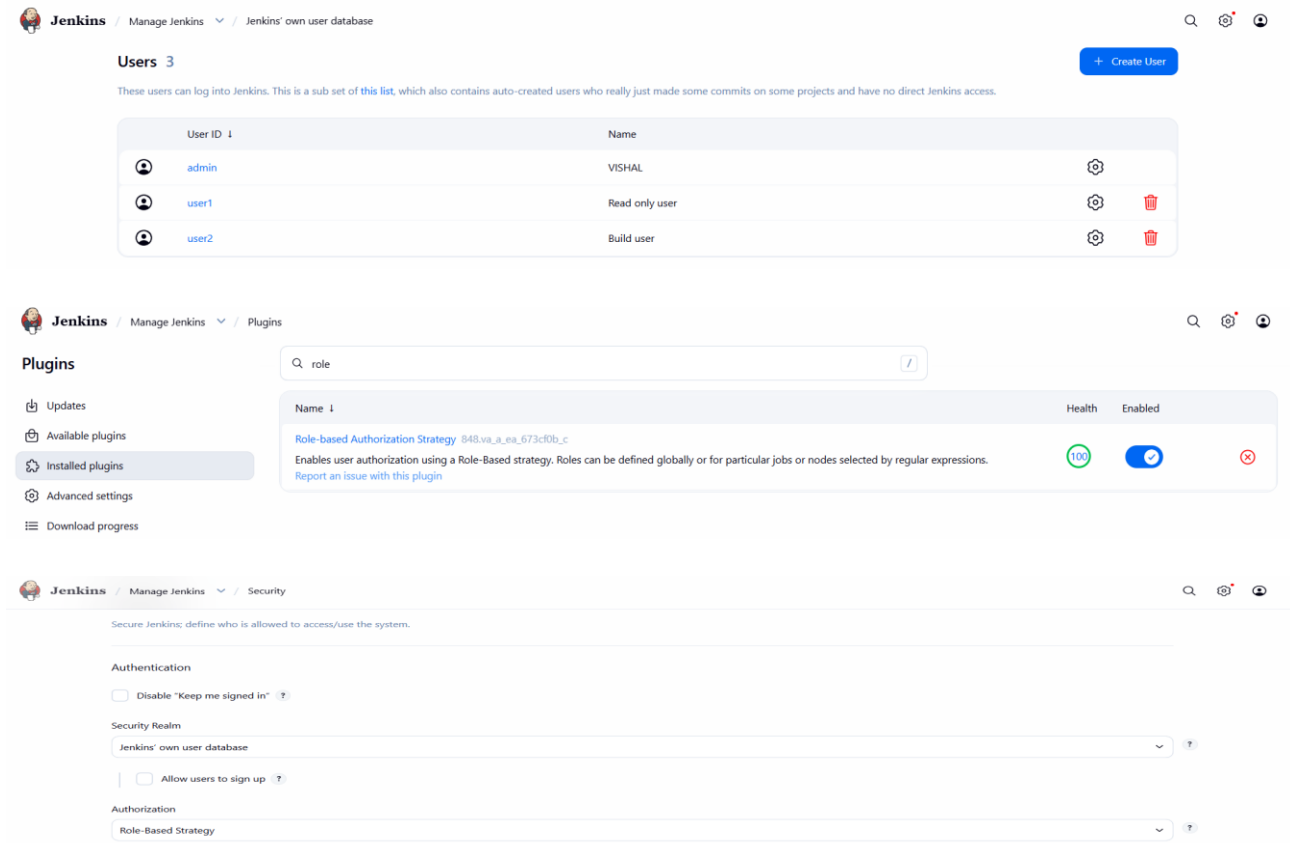
C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins>exit 0
Archiving artifacts
Finished: SUCCESS
```

TASK 9:

Users & Roles Objective: Manage Jenkins users

Tasks:

1. Create two users



The screenshot shows the Jenkins 'Manage Jenkins' page with the 'Users' tab selected. The page displays a table of users and a 'Create User' button.

User ID	Name	Health	Enabled
admin	VISHAL		
user1	Read only user		
user2	Build user		

Below the table, there is a 'Plugins' section with a search bar and a table of installed plugins. The 'Role-based Authorization Strategy' plugin is highlighted.

Name	Health	Enabled
Role-based Authorization Strategy 848.va_a_ea_673c10b_c	100	Enabled

The 'Security' section is also visible, showing the 'Authentication' and 'Authorization' settings. The 'Authentication' section is currently set to 'Jenkins' own user database'.

Jenkins / Manage Jenkins / Manage and Assign Roles / Manage Roles

Manage Roles

Assign Roles

Permission Templates

Role Strategy Macros

Global roles

Role	Overall	Credentials	Agent	Job	Run	View	SCM	Metrics
	Administrator	Read	Create	Delete	Update	Configure	Read	View
admin	☒							
build user		☒		☒				
read only		☒						

Role to add

Jenkins / Manage Jenkins / Manage and Assign Roles / Assign Roles

Manage Roles

Assign Roles

Permission Templates

Role Strategy Macros

Assign Roles

Global roles

User/Group	admin	build user	read only
Anonymous	☐	☐	☐
Authenticated Users	☐	☐	☐
VISHAL	☒	☐	☐
Read only user	☐	☐	☒
Build user	☐	☒	☐

Add User Add Group

2. Assign read-only permission to one user (**Read only user**)

Jenkins

Build History

All

S	W	Name	Last Success	Last Failure	Last Duration
☒	☒	Hello-Jenkins	18 min #12	2 hr 52 min #4	2.8 sec

Build Queue

Build Executor Status

Icon: S M L

3. Assign build permission to another user (**Built only user**)

Jenkins / Hello-Jenkins

Status

Changes

Build Now

Builds

Filter

Today

- #12 7:37 PM
- #11 6:24 PM
- #10 5:54 PM
- #9 5:43 PM
- #8 5:36 PM
- #7 5:32 PM
- #6 5:24 PM
- #5 5:05 PM
- #4 5:02 PM
- #3 5:02 PM
- #2 4:30 PM
- #1 4:29 PM

Permalinks

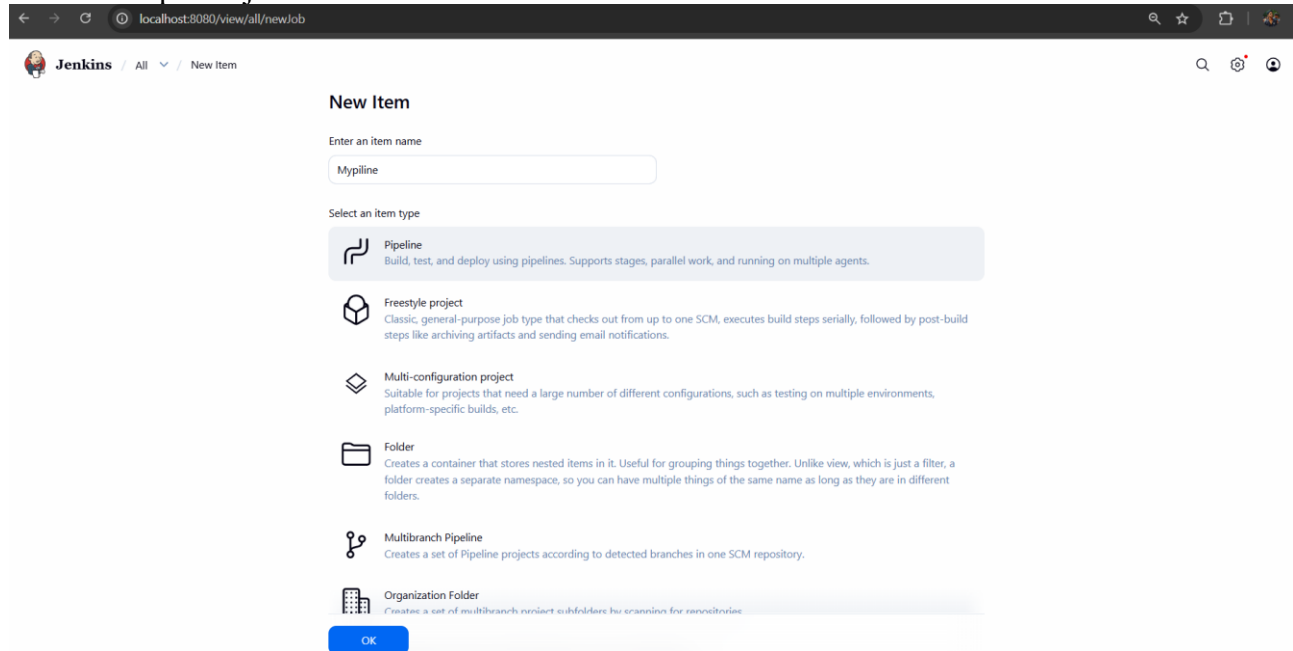
- Last build (#12), 19 min ago
- Last stable build (#12), 19 min ago
- Last successful build (#12), 19 min ago
- Last failed build (#4), 2 hr 53 min ago
- Last unsuccessful build (#4), 2 hr 53 min ago
- Last completed build (#12), 19 min ago

TASK 10:

Simple Jenkins Pipeline Objective: Create basic pipeline

Tasks:

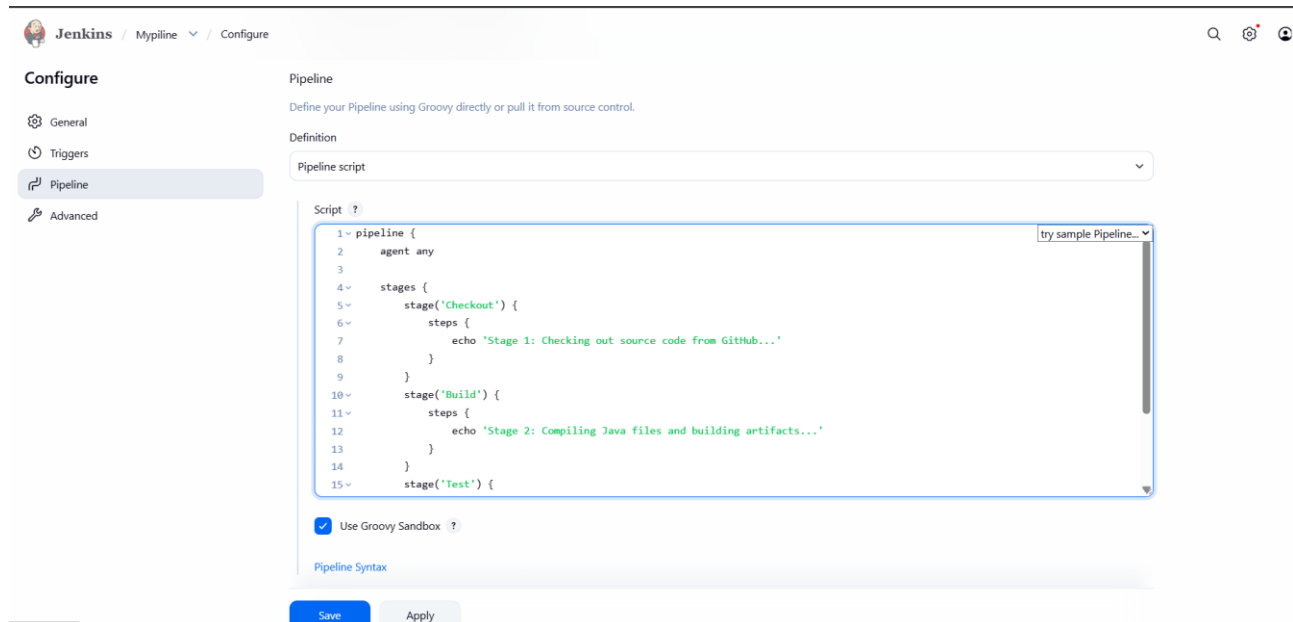
1. Create Pipeline job



The screenshot shows the Jenkins 'New Item' page. The browser address bar indicates the URL is localhost:8080/view/all/newJob. The page title is 'Jenkins / All / New Item'. The main heading is 'New Item'. Below this, there is a text input field for 'Enter an item name' with the value 'Mypipeline'. Underneath, there is a section 'Select an item type' with several options: 'Pipeline' (selected), 'Freestyle project', 'Multi-configuration project', 'Folder', 'Multibranch Pipeline', and 'Organization Folder'. Each option has a brief description. At the bottom of the 'Select an item type' section is a blue 'OK' button.

2. Write pipeline with stages:

- o Checkout
- o Build
- o Test

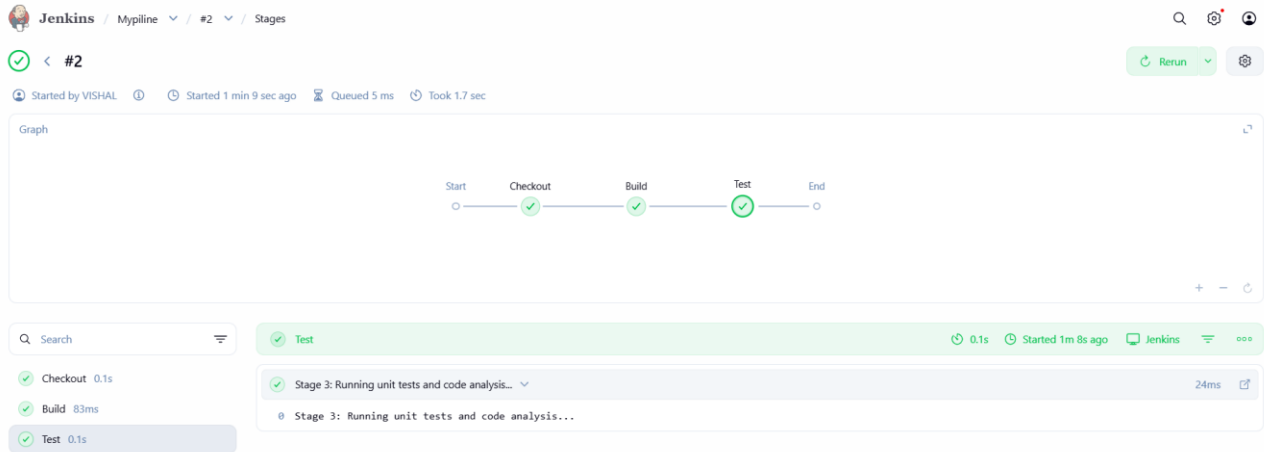


The screenshot shows the Jenkins 'Configure' page for the 'Mypipeline' job. The browser address bar indicates the URL is localhost:8080/view/all/newJob. The page title is 'Jenkins / Mypipeline / Configure'. The main heading is 'Configure'. On the left, there is a sidebar with navigation links: 'General', 'Triggers', 'Pipeline' (selected), and 'Advanced'. The main content area is titled 'Pipeline' and contains the text 'Define your Pipeline using Groovy directly or pull it from source control.' Below this, there is a dropdown menu for 'Definition' with the value 'Pipeline script'. Underneath, there is a text area for 'Script' with the following Groovy code:

```
1 ~ pipeline {
2   agent any
3
4   stages {
5     stage('Checkout') {
6       steps {
7         echo 'Stage 1: Checking out source code from GitHub...'
8       }
9     }
10    stage('Build') {
11      steps {
12        echo 'Stage 2: Compiling Java files and building artifacts...'
13      }
14    }
15    stage('Test') {
```

Below the script text area, there is a checkbox labeled 'Use Groovy Sandbox' which is checked. At the bottom of the page, there are two buttons: 'Save' and 'Apply'.

3. Run pipeline



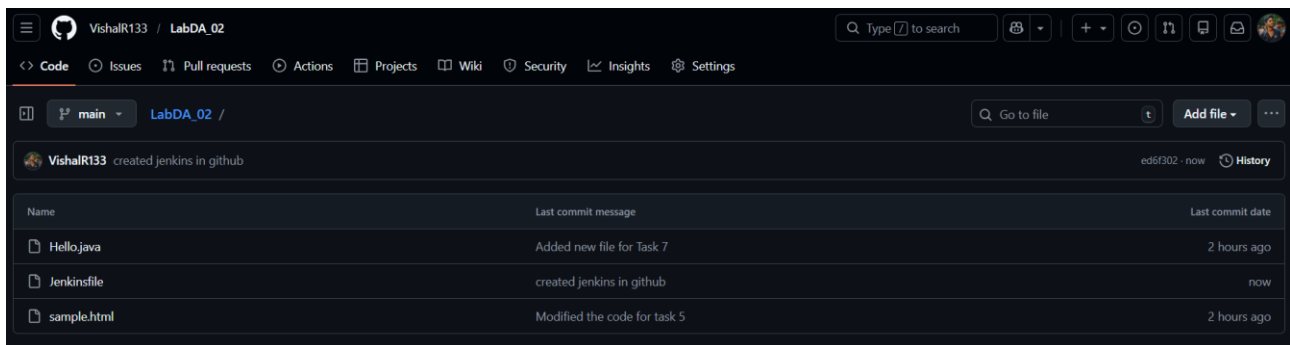
The screenshot shows the Jenkins interface for running pipeline #2. At the top, it indicates the pipeline was started by VISHAL, started 1 min 9 sec ago, queued for 5 ms, and took 1.7 sec to complete. A progress graph shows the pipeline stages: Start, Checkout, Build, Test, and End. The Test stage is currently selected and highlighted in green, showing a duration of 0.1s and a start time of 1m 8s ago. Below the graph, a list of stages is shown: Checkout (0.1s), Build (83ms), and Test (0.1s). The Test stage is expanded, showing the task 'Stage 3: Running unit tests and code analysis...' with a duration of 24ms.

TASK 11:

Jenkinsfile from Git Objective: Pipeline as Code

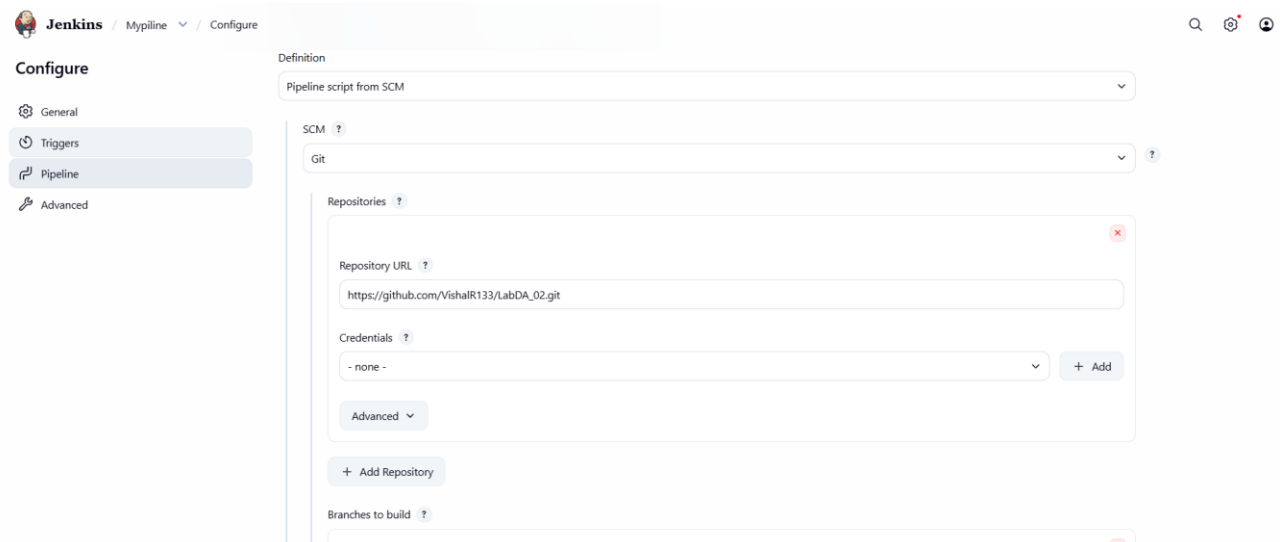
Tasks:

1. Create Jenkinsfile in Git repo



The screenshot shows the GitHub repository interface for 'VishalR133 / LabDA_02'. The repository is currently on the 'main' branch. A table of files is displayed, showing the commit history for each file. The files listed are 'Hello.java', 'Jenkinsfile', and 'sample.html'. The 'Jenkinsfile' was created by 'VishalR133' 2 hours ago, with the commit message 'created jenkins in github'. The 'Hello.java' file was added for 'Task 7' 2 hours ago. The 'sample.html' file was modified for 'task 5' 2 hours ago.

2. Configure pipeline from SCM



The screenshot shows the Jenkins 'Configure' page for the pipeline. The 'Definition' section is set to 'Pipeline script from SCM'. The 'SCM' is configured as 'Git'. The 'Repository URL' is set to 'https://github.com/VishalR133/LabDA_02.git'. The 'Credentials' are set to '- none -'. The 'Advanced' section is expanded, showing the 'Add Repository' button. The 'Branches to build' section is also visible at the bottom.

3. Trigger build

The screenshot shows the Jenkins console output for a pipeline build. The left sidebar contains navigation links: Status, Changes, Console Output (selected), Edit Build Information, Delete build '#3', Timings, Git Build Data, Pipeline Overview, Restart from Stage, Replay, Pipeline Steps, Workspaces, and Previous Build. The main console area displays the following log:

```
Started by user VISHAL
Obtained Jenkinsfile from git https://github.com/VishalR133/Lab0A_02.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\workspace\Mypipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/VishalR133/Lab0A_02.git
> git.exe init C:\ProgramData\Jenkins\workspace\Mypipeline # timeout=10
Fetching upstream changes from https://github.com/VishalR133/Lab0A_02.git
> git.exe --version # timeout=10
> git --version # 'git version 2.52.0.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/VishalR133/Lab0A_02.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe config remote.origin.url https://github.com/VishalR133/Lab0A_02.git # timeout=10
> git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git.exe rev-parse "refs/remotes/origin/main^[commit]" # timeout=10
Checking out Revision ed6f3023e75a17630f6dc82f73b5a7632d533273 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f ed6f3023e75a17630f6dc82f73b5a7632d533273 # timeout=10
Commit message: "created jenkins in github"
First time build. Skipping changelog.
Finished: SUCCESS
```

The screenshot shows the Jenkins build overview for stage #3. The top bar indicates the build was started by VISHAL, started 2 min 26 sec ago, queued 5 ms, and took 8.4 sec. The build status is green with a checkmark. The left sidebar shows a search bar and a list of build steps: Checkout SCM (3s), Checkout (94ms), Build (70ms), and Test (0.1s). The main area displays a graph of the build process with steps: Start, Checkout SCM, Checkout, Build, Test, and End. The Test step is highlighted in green. Below the graph, the Test step details are shown: Testing from Jenkinsfile... (27ms).

TASK 12:

Post-Build Actions

Objective: Handle build result

Tasks:

1. Add post section

The screenshot shows a GitHub repository interface. The top bar displays the repository name 'main', 1 Branch, 0 Tags, a search bar, and buttons for 'Add file' and 'Code'. The main content area shows a commit by VishalR133 titled 'Updated the file for task 12' with commit hash e7e6ac2 and 6 Commits. Below the commit, a list of files is shown:

File	Commit Message	Time
Hello.java	Added new file for Task 7	2 hours ago
Jenkinsfile	Updated the file for task 12	now
sample.html	Modified the code for task 5	3 hours ago

2. Print message on success/failure

The screenshot shows the Jenkins console output for a pipeline named 'Mypline'. The output details the steps of the pipeline, including checking out code from a Git repository, building, and testing. The pipeline concludes with a success message: 'SUCCESS: The build and tests finished perfectly!'.

```
Started by user VISHAL
Obtained Jenkinsfile from git https://github.com/VishalR133/Lab0A_02.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\jenkins\workspace\Mypline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\jenkins\workspace\Mypline\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/VishalR133/Lab0A_02.git # timeout=10
Fetching upstream changes from https://github.com/VishalR133/Lab0A_02.git
> git.exe --version # timeout=10
> git --version # 'git version 2.52.0.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/VishalR133/Lab0A_02.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision e7e6ac2ff368d8c877155c4b40424551cde2fa71 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f e7e6ac2ff368d8c877155c4b40424551cde2fa71 # timeout=10
Commit message: "Updated the file for task 12"
> git.exe rev-list --no-walk ed6f3023e75a17630f6dc82f73b5a7632d533273 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] echo
Checking out code...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Testing...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
The pipeline execution is now complete.
[Pipeline] echo
SUCCESS: The build and tests finished perfectly!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

TASK 13:

Trigger Job from Another Job

Objective: Job chaining

Tasks:

1. Create Job-A and Job-B

The screenshot shows the Jenkins dashboard with a list of jobs. The jobs are 'Hello-Jenkins', 'Job_A', 'Job_B', and 'Mypline'. The 'Mypline' job is highlighted, indicating it is the current job being viewed.

S	W	Name	Last Success	Last Failure	Last Duration
🟢	🟡	Hello-Jenkins	54 min #12	3 hr 28 min #4	2.8 sec
🟡	🟡	Job_A	N/A	N/A	N/A
🟡	🟡	Job_B	N/A	N/A	N/A
🟢	🟡	Mypline	6 min 39 sec #4	N/A	9.7 sec



Jenkins / Job_A ▾ / Configure

Configure

⚙️ General

🔑 Source Code Management

🕒 Triggers

🌐 Environment

📋 Build Steps

📦 Post-build Actions

- ☐ Add timestamps to the Console Output
- ☐ Inspect build log for published build scans
- ☐ Terminate a build if it's stuck
- ☐ With Ant ?

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

⚙️ Execute Windows batch command ?

Command

See [the list of available environment variables](#)

```
echo "This is Job_A running!!....."
```

Advanced ▾

+ Add build step

Post-build Actions

Save

Apply



Jenkins / Job_B ▾ / Configure

Configure

⚙️ General

🔑 Source Code Management

🕒 Triggers

🌐 Environment

📋 Build Steps

📦 Post-build Actions

- ☐ Delete workspace before build starts
- ☐ Use secret text(s) or file(s) ?
- ☐ Add timestamps to the Console Output
- ☐ Inspect build log for published build scans
- ☐ Terminate a build if it's stuck
- ☐ With Ant ?

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

⚙️ Execute Windows batch command ?

Command

See [the list of available environment variables](#)

```
echo "This is Job_B running after Job_A!!"
```

Advanced ▾

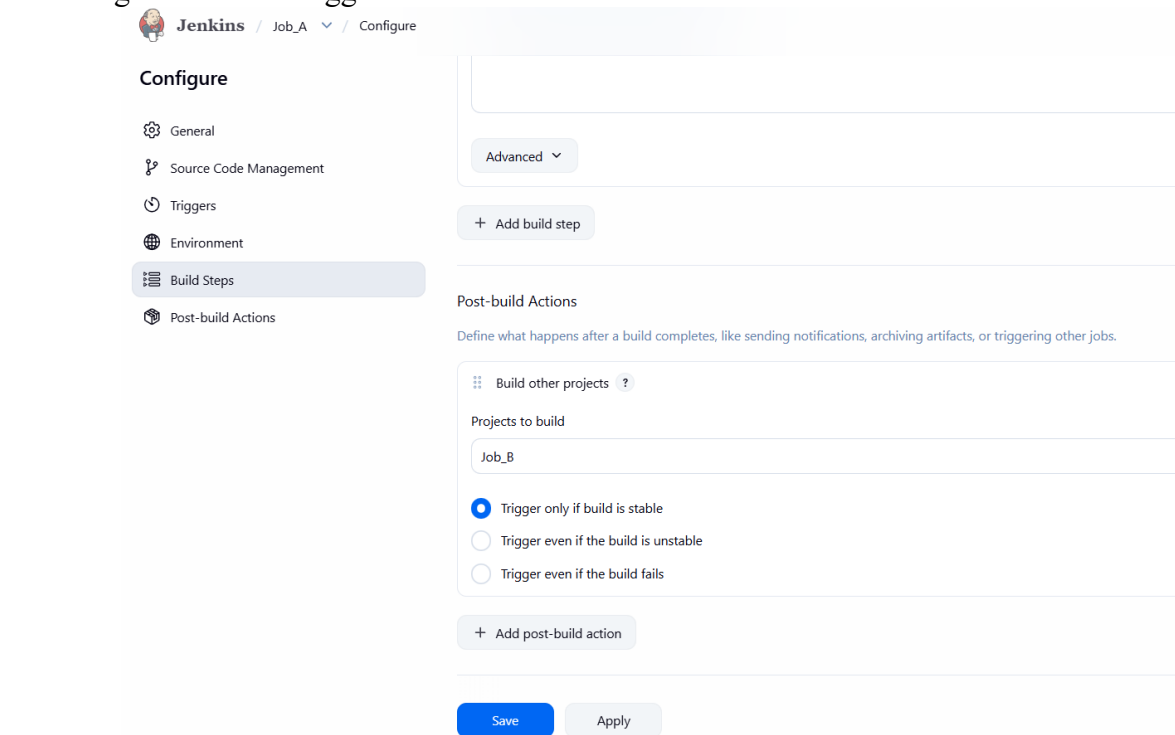
+ Add build step

Save

Apply

localhost:8080

2. Configure Job-B to trigger after Job-A



The screenshot shows the Jenkins configuration page for Job_A, specifically the 'Build Steps' tab. The left sidebar lists configuration options: General, Source Code Management, Triggers, Environment, Build Steps (selected), and Post-build Actions. The main area shows the 'Advanced' dropdown and a '+ Add build step' button. Below this, the 'Post-build Actions' section is visible, with a description: 'Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.' Under 'Build other projects', the 'Projects to build' field contains 'Job_B'. The 'Trigger only if build is stable' radio button is selected. At the bottom, there are 'Save' and 'Apply' buttons.



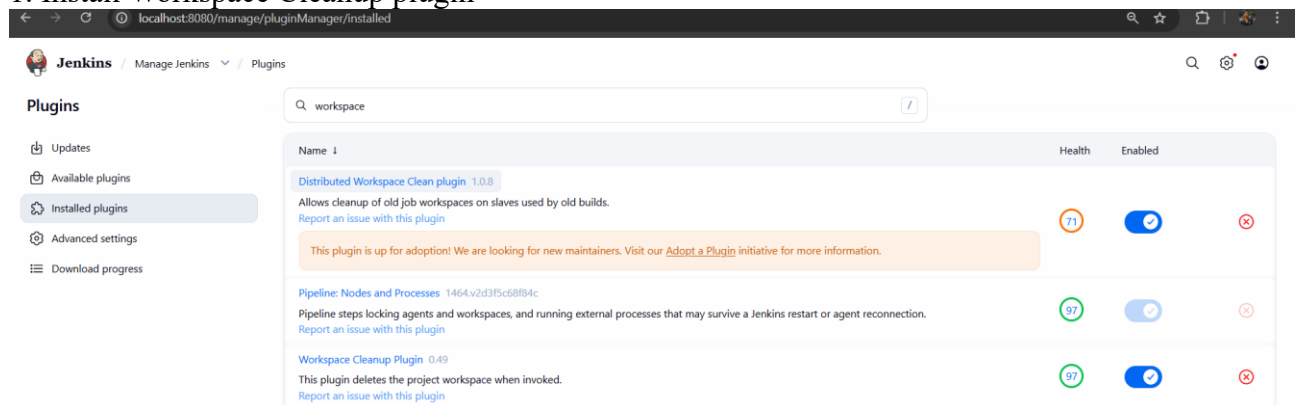
The screenshot shows the Jenkins console output for Job_B. The left sidebar lists options: Status, Changes, Console Output (selected), Edit Build Information, Delete build '#1', and Timings. The console output shows the build was started by user VISHAL, running as SYSTEM, and building in workspace C:\ProgramData\Jenkins\jenkins\workspace\Job_B. The console log includes the command 'cmd /c call C:\windows\TEMP\jenkins3493470097993916287.bat' and the output 'This is Job_B running after Job_A!!'. The build finished successfully.

TASK 14:

Workspace Cleanup Objective: Manage disk usage

Tasks:

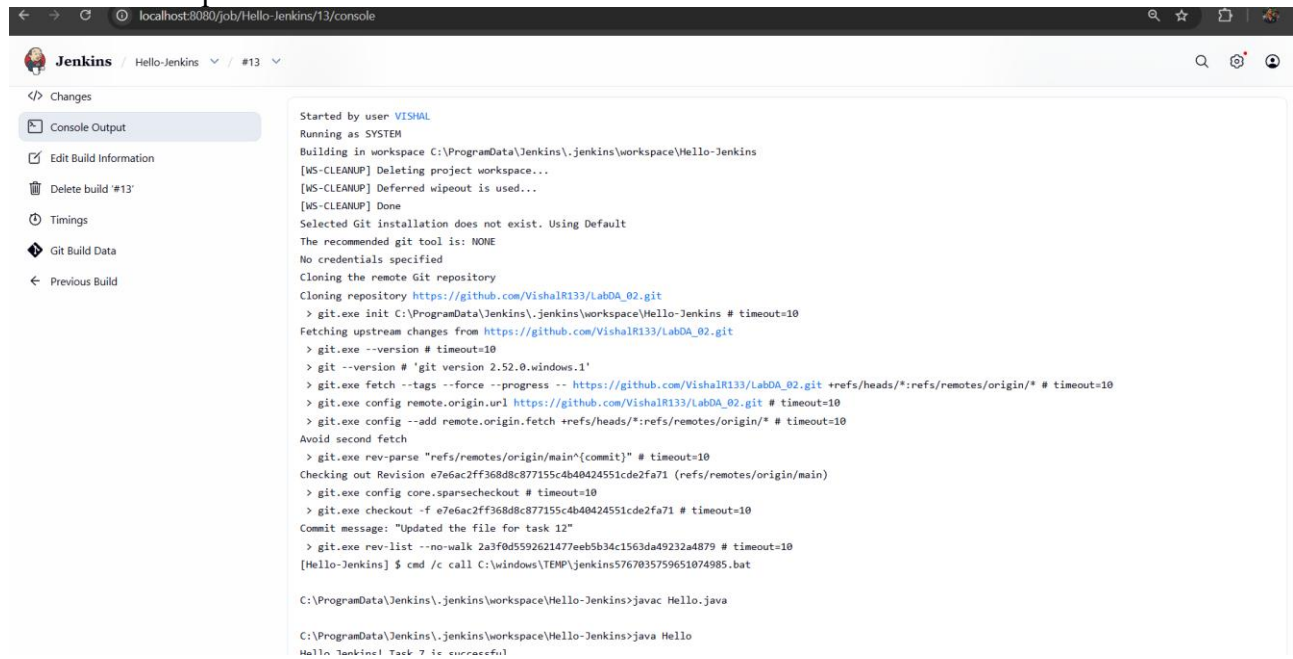
1. Install Workspace Cleanup plugin



The screenshot shows the Jenkins 'Manage Jenkins' page, specifically the 'Plugins' tab. The search bar contains 'workspace'. The table lists three plugins:

Name	Health	Enabled
Distributed Workspace Clean plugin 1.0.8 Allows cleanup of old job workspaces on slaves used by old builds. Report an issue with this plugin	71	☒
Pipeline: Nodes and Processes 1464.v2d3f5c6884c Pipeline steps locking agents and workspaces, and running external processes that may survive a Jenkins restart or agent reconnection. Report an issue with this plugin	97	☒
Workspace Cleanup Plugin 0.49 This plugin deletes the project workspace when invoked. Report an issue with this plugin	97	☒

2. Clean workspace before build



```
Started by user VISHAL
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins
[WS-CLEANUP] Deleting project workspace...
[WS-CLEANUP] Deferred wipeout is used...
[WS-CLEANUP] Done
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/VishalR133/LabDA_02.git
> git.exe init C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins # timeout=10
Fetching upstream changes from https://github.com/VishalR133/LabDA_02.git
> git.exe --version # timeout=10
> git --version # 'git version 2.52.0.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/VishalR133/LabDA_02.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe config remote.origin.url https://github.com/VishalR133/LabDA_02.git # timeout=10
> git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision e7e6ac2ff368d8c877155c4b40424551cde2fa71 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f e7e6ac2ff368d8c877155c4b40424551cde2fa71 # timeout=10
Commit message: "Updated the file for task 12"
> git.exe rev-list --no-walk 2a3f0d5592621477eeb5b34c1563da49232a4879 # timeout=10
[Hello-Jenkins] $ cmd /c call C:\windows\TEMP\jenkins5767035759651074985.bat

C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins>javac Hello.java

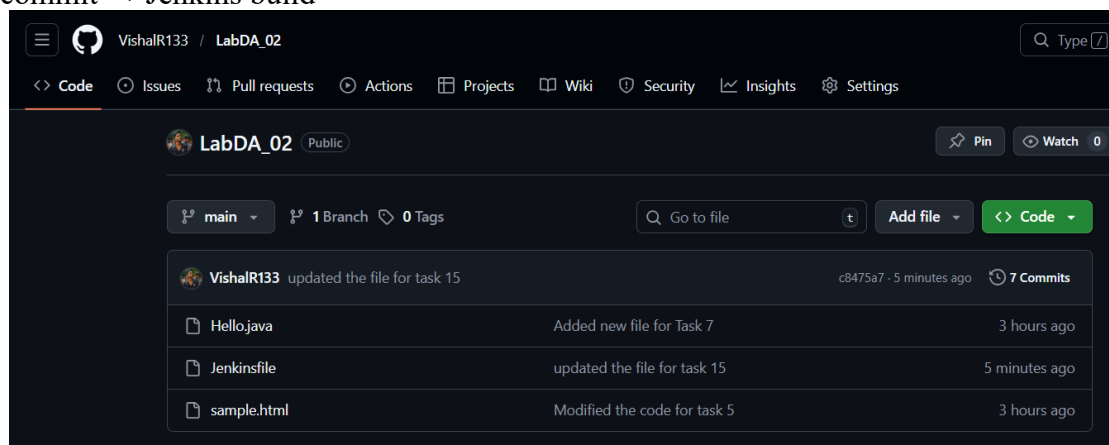
C:\ProgramData\Jenkins\jenkins\workspace\Hello-Jenkins>java Hello
Hello Jenkins! Task 7 is successful
```

TASK 15:

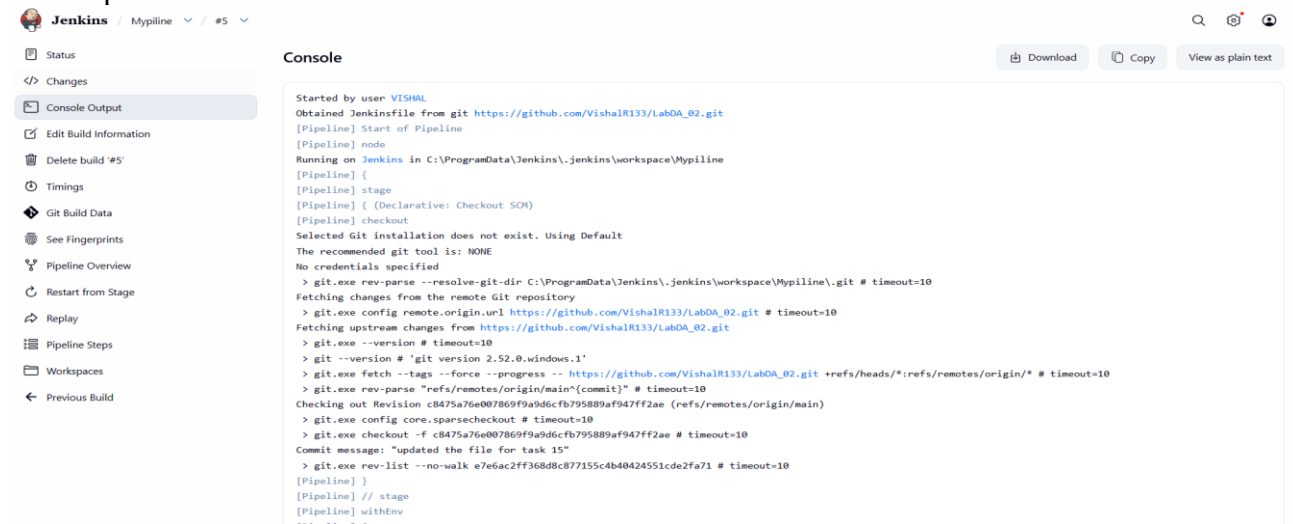
Mini CI Project Objective: Implement basic CI flow

Tasks:

1. Git commit → Jenkins build



2. Compile code



```
Started by user VISHAL
Obtained Jenkinsfile from git https://github.com/VishalR133/LabDA_02.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\jenkins\workspace\Mypipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\jenkins\workspace\Mypipeline\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/VishalR133/LabDA_02.git # timeout=10
Fetching upstream changes from https://github.com/VishalR133/LabDA_02.git
> git.exe --version # timeout=10
> git --version # 'git version 2.52.0.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/VishalR133/LabDA_02.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision c8475a76e007869f9a9dcfb795889af947ff2ae (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f c8475a76e007869f9a9dcfb795889af947ff2ae # timeout=10
Commit message: "updated the file for task 15"
> git.exe rev-list --no-walk e7e6ac2ff368d8c877155c4b40424551cde2fa71 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
  ...
```

```

[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Compile)
[Pipeline] echo
Compiling Java code...
[Pipeline] bat

C:\ProgramData\Jenkins\jenkins\workspace\Mypline>javac Hello.java
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Archive)
[Pipeline] echo
Archiving build artifacts...
[Pipeline] archiveArtifacts
Archiving artifacts
Recording fingerprints
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
CI Build Successful! Artifacts are ready.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

Jenkins / Mypline / #5 / Stages

Started by VISHAL · Started 1 min 54 sec ago · Queued 5 ms · Took 15 sec · <> Changes · Artifacts

Graph

```

graph LR
    Start((Start)) --> CheckoutSCM((Checkout SCM))
    CheckoutSCM --> Checkout((Checkout))
    Checkout --> Compile((Compile))
    Compile --> Archive((Archive))
    Archive --> PostActions((Post Actions))
    PostActions --> End((End))

```

Search

- Checkout SCM 3s
- Checkout 2s
- Compile 2s
- Archive 0.2s
- Post Actions 86ms

Post Actions

- CI Build Successful! Artifacts are ready. 32ms

3. Archive artifacts

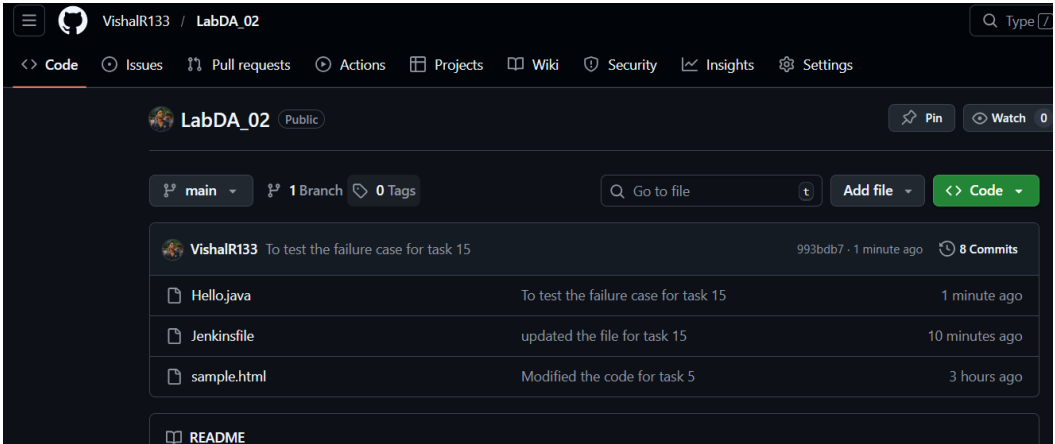
```

localhost:8080/job/Mypline/5/consoleText

Started by user VISHAL
Obtained Jenkinsfile from git https://github.com/VishalR133/LabDA_02.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\jenkins\workspace\Mypline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\jenkins\workspace\Mypline\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/VishalR133/LabDA_02.git # timeout=10
Fetching upstream changes from https://github.com/VishalR133/LabDA_02.git
> git.exe --version # timeout=10
> git --version # 'git version 2.52.0.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/VishalR133/LabDA_02.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse 'refs/remotes/origin/main^{commit}' # timeout=10
Checking out Revision c8475a76e0b7869f9a9dcfb795889af947ff2ae (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f c8475a76e0b7869f9a9dcfb795889af947ff2ae # timeout=10
Commit message: "updated the file for task 15"
> git.exe rev-list --no-walk e7e6ac2ff36d8c877155c4b40424551cde2fa71 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] echo
Pulling latest code from Github...
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\jenkins\workspace\Mypline\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/VishalR133/LabDA_02.git # timeout=10
Fetching upstream changes from https://github.com/VishalR133/LabDA_02.git
> git.exe --version # timeout=10
> git --version # 'git version 2.52.0.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/VishalR133/LabDA_02.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse 'refs/remotes/origin/main^{commit}' # timeout=10
Checking out Revision c8475a76e0b7869f9a9dcfb795889af947ff2ae (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f c8475a76e0b7869f9a9dcfb795889af947ff2ae # timeout=10
Commit message: "updated the file for task 15"
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage

```

4. Fail build on error



Jenkins / Mypline / #6 / Stages

#6 Started by VISHAL Started 20 sec ago Queued 4 ms Took 13 sec Changes Rerun Settings

Graph

Start → Checkout SCM (✓) → Checkout (✓) → **Compile (✗)** → Archive (⏸) → Post Actions (✓) → End

Compile 2s Started 10s ago Jenkins

- ✓ Compiling Java code... 37ms
- ✗ Windows Batch Script `javac Hello.java` 2s
 - 0
 - 1 C:\ProgramData\Jenkins\jenkins\workspace\Mypline>javac Hello.java
 - 2 Hello.java:3: error: ';' expected
 - 3 System.out.println("Hello Jenkins! Task 7 is successful.")
 - 4
 - 5 1 error
 - 6 script returned exit code 1

CI TOOL: WOODPECKER CI

Companies and Organizations Using Woodpecker CI :

Woodpecker CI is a self-hosted, open-source Continuous Integration tool. It is mainly adopted by open-source communities, startups, and organizations that prefer full control over their CI/CD infrastructure. Some known organizations and types of companies using Woodpecker CI are listed below:

Organizations Using Woodpecker CI

1. Codeberg
2. Woodpecker CI (used for self-testing and development)

Types of Companies and Teams Using Woodpecker CI

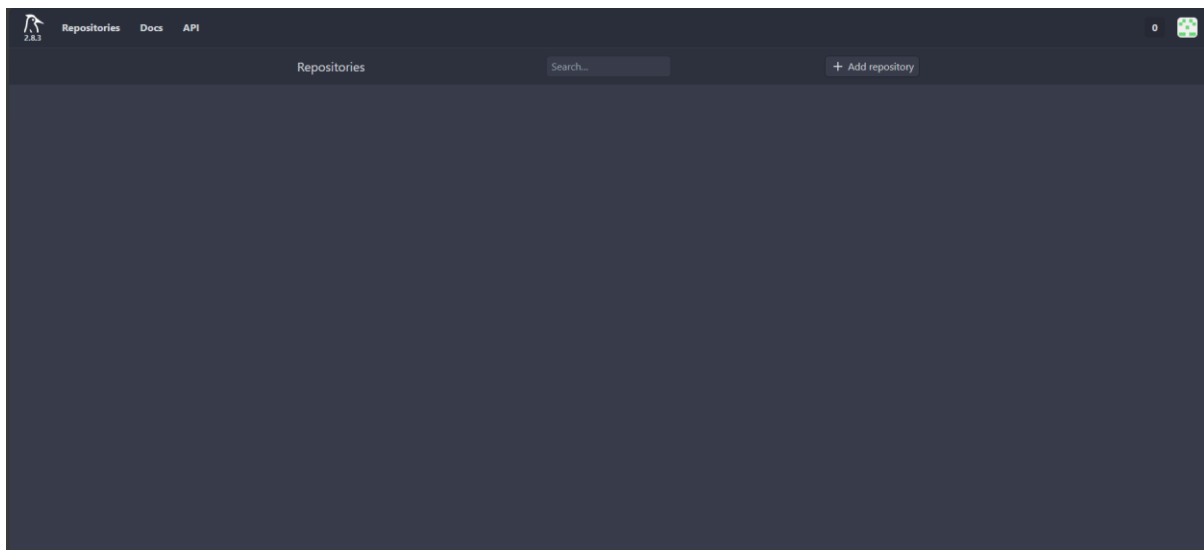
1. Open-source software communities
2. Startups using self-hosted DevOps tools
3. Small and medium-scale software companies
4. Research and academic institutions
5. DevOps-focused engineering teams
6. Organizations preferring on-premise CI/CD solutions

TASK 1:

1) Look at the main page after login

2) Identify:

- Left side menu
- Dashboard / Home
- Repositories option



TASK 2:
Woodpecker version

```
PS C:\Users\yashm\OneDrive\Desktop\woodpecker> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
73c3e8e2ad08	woodpeckerci/woodpecker-server:latest	"/bin/woodpecker-ser..."	7 minutes ago	Up 7 minutes (healthy)
0.0.0.0:8000->8000/tcp		woodpecker-woodpecker-server-1		

TASK 3:

← VishalR133 / hello-woodpecker

Tasks

 Warnings (1)

Config

 VishalR133

 main

 72c75fdb58

 clone

 hello

TASK 4:
Git integration

VishalR133 / hello-woodpecker

piposine

ntare



Activity

Branches

Pull requests

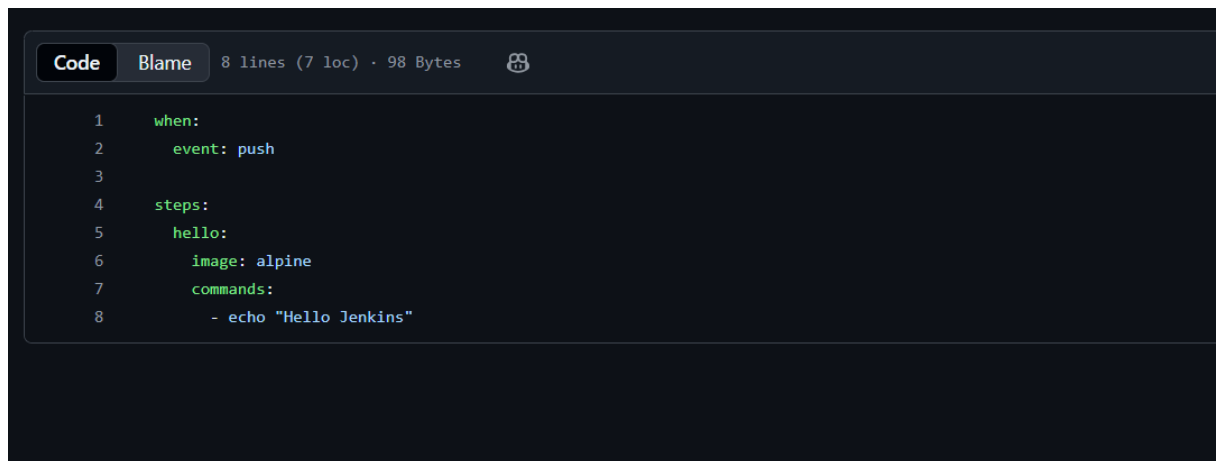
Run pipeline

main

default

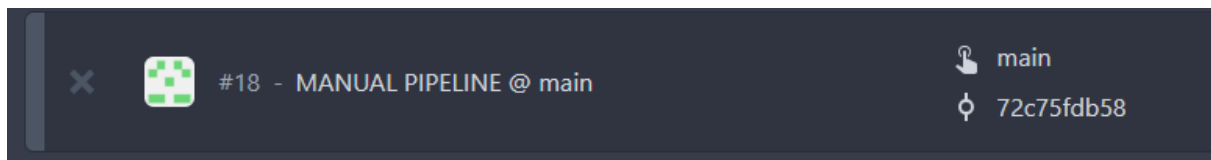
TASK 5:

Automatic build



```
Code Blame 8 lines (7 loc) · 98 Bytes
```

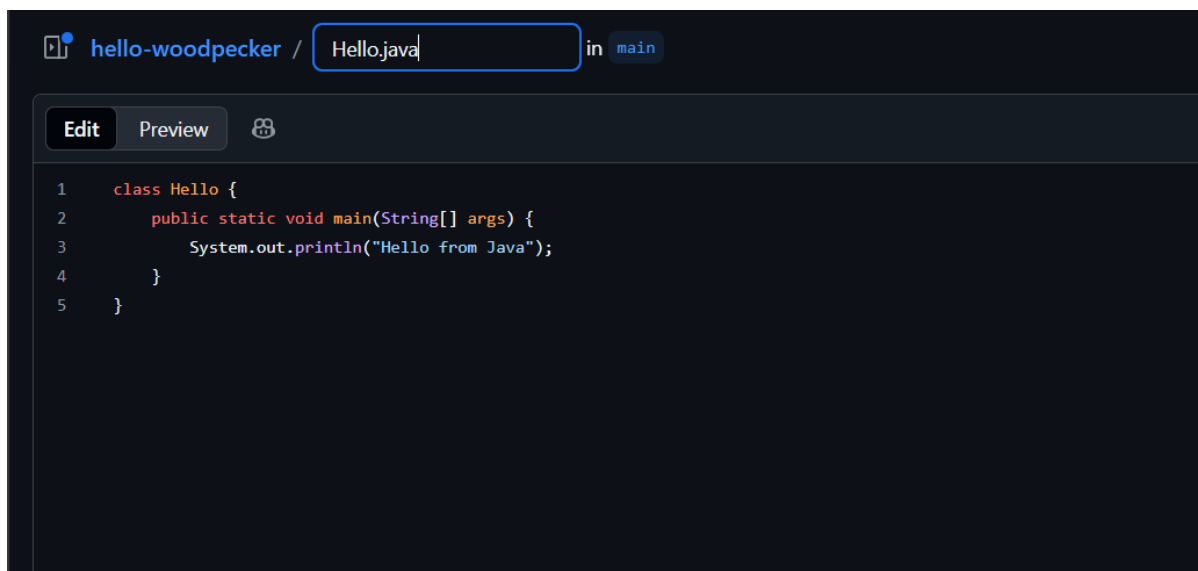
```
1  when:
2    event: push
3
4  steps:
5    hello:
6      image: alpine
7      commands:
8        - echo "Hello Jenkins"
```



TASK 6:

Parameterized Build

Showed multiple errors for the paramatereized build



```
hello-woodpecker / Hello.java in main
```

```
Edit Preview
```

```
1  class Hello {
2    public static void main(String[] args) {
3      System.out.println("Hello from Java");
4    }
5  }
```

← VishalR133 / hello-woodpecker

Tasks

 Warnings (1)

Config

 VishalR133

 main

 72c75fdb58

 clone

 hello

TASK 7: artifacts

Code

Blame

9 lines (8 loc) · 121 Bytes



```
1  steps:
2    java-build:
3      image: openjdk:17
4      commands:
5        - javac Hello.java
6
7  artifacts:
8    paths:
9      - Hello.class
```

Activity

Branches

Pull requests

Run pipeline



 #31 - MANUAL PIPELINE @ main

 main

 5b7ed584ff

 not started yet

 a few seconds ago